Visualizador de un UR3e por TCP/IP

Memoria de la práctica · código en github.com/godzilin/Practica1UR

1. Qué hace

Lee por la red el estado de un brazo **UR3e** (real o simulado con URSim) y lo reproduce en 3D con **PyBullet**. Encima, una mini-aplicación permite **enseñar posturas** al robot y repetirlas.

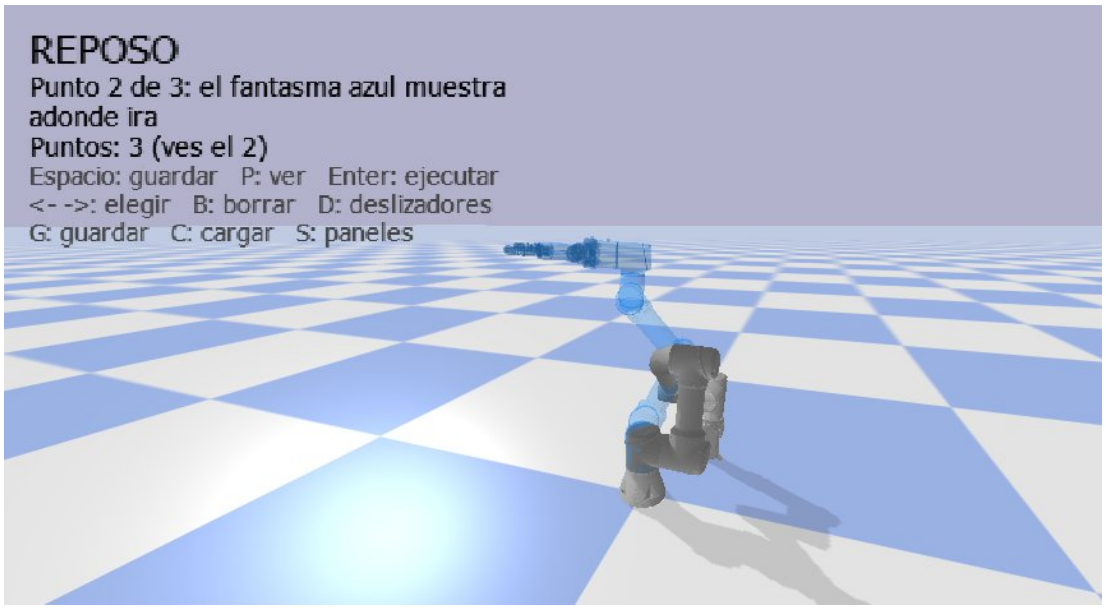


Fig. 1 · Aplicación conectada a URSim. Gris: el robot. Azul: el punto que va a ejecutar («fantasma»).

2. Arquitectura

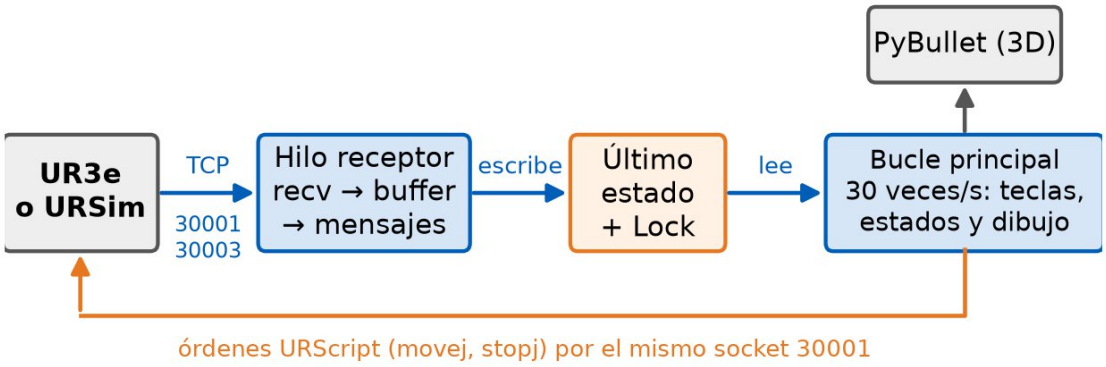


Fig. 2 · Quién hace qué: recibir, guardar el último estado y dibujar.

- **Un hilo por robot solo recibe.** Si un robot se cuelga, solo se bloquea su hilo.
- **Un único bucle dibuja y envía órdenes.** PyBullet nunca se usa desde dos hilos.
- **Lock** en el estado compartido: nunca se lee un estado a medio escribir.

3. Leer un flujo TCP

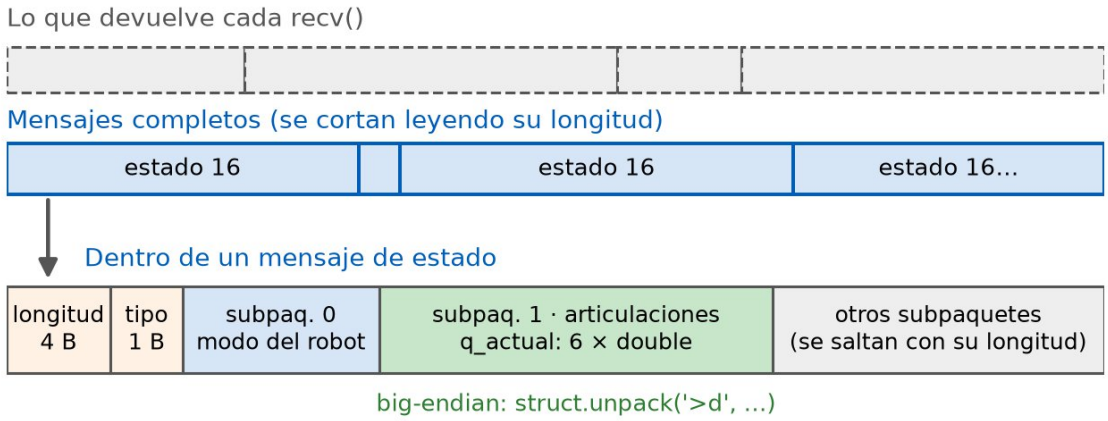


Fig. 3 · De los bytes que llegan por TCP a los 6 ángulos del robot.

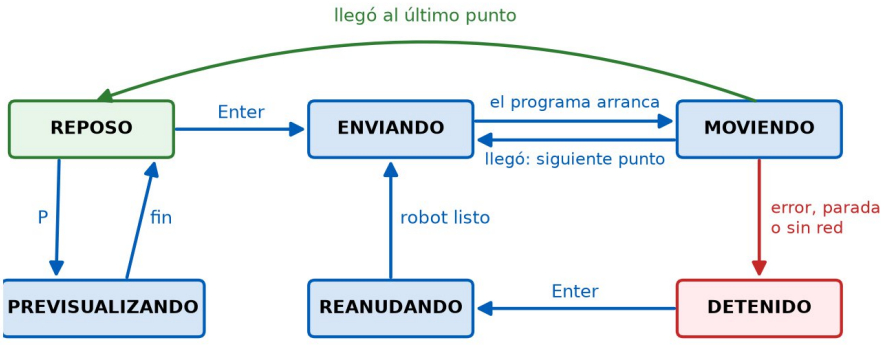
- **Un recv() no es un mensaje:** en un minuto hubo 1110 `recv()` de 11 tamaños distintos. Se acumula en un buffer y se corta leyendo la longitud de cada mensaje.
- **Subpaquetes que no interesan:** se saltan sumando su longitud, sin entenderlos.
- **Big-endian:** el robot ordena los bytes al revés que el PC; se indica con `>`.

4. Las cuatro tareas

Tarea	Script	Idea clave
A · Primaria	`primaria.py`	Puerto 30001 · 10 estados/s · ángulos del subpaquete 1
B · Tiempo real	`tiempo_real.py`	Puerto 30003 · ~500 paquetes/s · se queda el último y dibuja 30/s
C · Varios robots	`varios_robots.py`	Un hilo por robot · «SIN DATOS» en rojo si uno se cae
D · Enseñar y reproducir	`ensenar_reproducir.py`	Enseñar · previsualizar · ejecutar y confirmar

5. Enseñar y reproducir

Una **máquina de estados** lleva la secuencia. El robot no responde a un `movej`: hay que deducir del estado que publica si lo ha cumplido.



Esc en cualquier momento: stopj y vuelta a REPOSO

Fig. 4 · Máquina de estados de la secuencia.

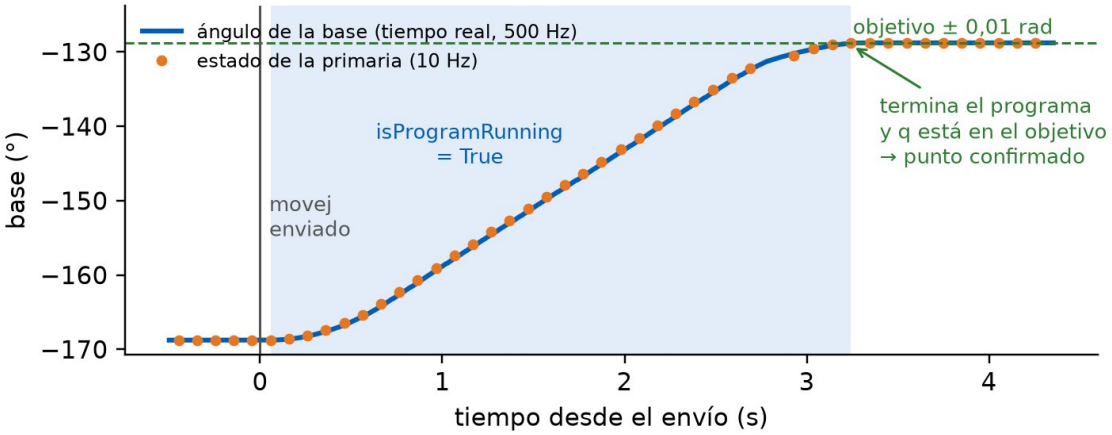


Fig. 5 · Datos reales de URSim. Un punto se da por bueno cuando el programa termina y el ángulo está a menos de 0,01 rad del objetivo.

6. Además

- **Repositorio GitHub** (github.com/godzilin/Practica1UR): cada cambio es un commit; README con instalación y uso.
- **A prueba de fallos:** reconexión automática, mensajes corruptos descartados, configuración validada. Probado con 20 000 mensajes rotos, cortes de red y URSim reiniciado.
- **Seguridad:** `stopj` si el programa se cierra con el robot en marcha; velocidad ≤ 0,3 rad/s; una parada de emergencia nunca se desbloquea por red.
- **Fácil de usar:** F5 en VS Code, ajustes como constantes al principio, IP en `robots.json`.

7. Robot real

[Completar tras la sesión: qué UR3e se usó y diferencias con URSim.]